

2.1 Linear Models for Regression

Squared Error in Linear Regression

1. The Core Regression Problem

In supervised learning, regression deals with predicting a continuous, real-valued target variable, $T \in \mathbb{R}$, based on some input variables X .

- **The Goal:** We want to learn a prediction function $y(X)$ such that $y(X) \approx T$ on average.
- **Parametric Regression:** Instead of remembering the whole dataset, we assume the function has a specific mathematical form governed by a fixed-dimensional parameter vector, $w \in \mathbb{R}^p$. Our job is to find the best w .

2. Measuring Success: Risk and Error

To find the best parameters, we need a way to measure how "wrong" our model is. This is done using a loss function, l .

- **Training Error (Empirical Risk):** This is the average loss calculated over the data you actually have (your training set). For a dataset of size N , the Mean Squared Error (MSE) is:
$$E(w) = \frac{1}{N} \sum_{n=1}^N \frac{1}{2} (t_n - y(x_n, w))^2$$
- **Generalization Error (Prediction Risk):** This is what we *actually* care about. It is the expected loss on new, unseen data.
- **The LOTUS Principle:** To compute the expected squared error, we use the Law of the Unconscious Statistician (LOTUS). Because the squared error itself is a random variable, LOTUS allows us to compute its expectation using the probability distribution of X , rather than having to derive a whole new probability distribution for the error itself.
- Simple terms: we use the probability distribution of X to compute the expected error, utilizing that X is a Random Variable

For continuous X we can write generalisation error, i.e., **expected squared error**, as follows

$$\mathbf{E}\left((T - y(X))^2\right) = \int_{D_X} \int_{D_T} (t - y(x))^2 p(x, t) dt dx$$

Why? The squared error $(T - y(X))^2 = Z$ is a new **random variable**. Don't we have to find its **density** p_Z to compute

$$\mathbf{E}(Z) = \int_{D_Z} z p_Z(z) dz$$

Recall **definition** for numerical random variable X :

$$\mathbf{E}[X] = \int_{D_X} x p(x) dx$$

continuous X

$$\mathbf{E}[X] = \sum_{x \in D_X} x p(x)$$

discrete X

Let f be some **function**. Then can compute expectation of $f(X)$ as follows:

$$\mathbf{E}[f(X)] = \int_{D_X} f(x) p(x) dx$$

this is a new
random variable

$$\mathbf{E}[f(X)] = \sum_{x \in D_X} f(x) p(x)$$

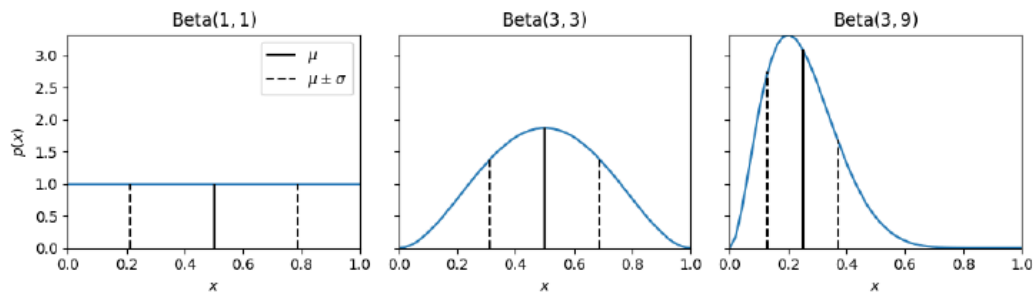
but can just use density or
mass function of X to compute
expectation

Variance and standard deviation of rv X measure “dispersion” of probability mass or density around its mean $\mu = E[X]$:

$$\text{Var}[X] = E[(X - \mu)^2] = \int_{D_X} (x - \mu)^2 p(x) dx \quad \text{or} \quad = \sum_{x \in D_X} (x - \mu)^2 p(x)$$

$$\text{Std}[X] = \sqrt{\text{Var}[X]} = \sigma$$

expected squared error of the mean
when predicting value for X



3. The Probabilistic Perspective: Why Squared Error?

You might wonder: *why do we default to squaring the errors?* Why not take the absolute value, or the error to the power of 4? The answer lies in statistics.

Let's assume our target variable T is generated by our deterministic function $y(X)$, but corrupted by some random noise, ϵ :

$$T = y(X) + \epsilon$$

- **The Central Limit Theorem (CLT):** If we assume that the noise ϵ comes from many small, independent, and identically distributed (i.i.d.) sources, the CLT tells us that the sum of these noises will converge to a Normal (Gaussian) distribution as the number of sources grows.
- Therefore, it is mathematically sound to assume our noise is normally distributed with a mean of 0 and a variance of σ^2 : $\epsilon \sim N(0, \sigma^2)$.

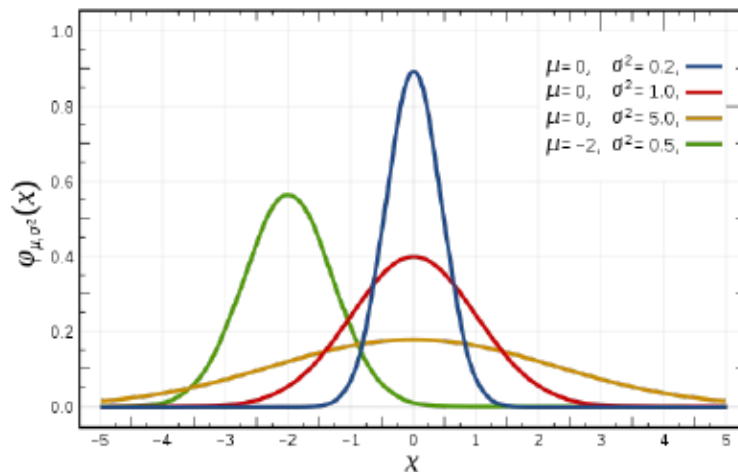
$$p(\epsilon) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{\epsilon^2}{2\sigma^2}\right)$$

normal distribution
 $\epsilon \sim N(0, \sigma^2)$

- for $\epsilon_1 \dots \epsilon_n$ for a big n

Because the noise is Gaussian, the probability of observing our target t , given the input x and parameters w , is also a Gaussian distribution centered around our prediction $y(x, w)$:

$$p(t|x, w) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(t-y(x, w))^2}{2\sigma^2}\right)$$



4. The Grand Unification: MLE = SSE

Probabilistic model

$$p(t | x, w) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(t - y(x, w))^2}{2\sigma^2}\right)$$

Log likelihood (of individual data point)

$$\ln p(t | x, w) = \underbrace{-\ln \sqrt{2\pi}\sigma - \frac{1}{\sigma^2}}_{\text{constants}} \underbrace{\frac{1}{2}(t - y(x, w))^2}_{\text{squared error}}$$

⇒ squared error minimisation is maximum likelihood estimation with additive normal (Gaussian) noise model

Here is where the magic happens. In statistics, a common way to find the best parameters is **Maximum Likelihood Estimation (MLE)**—finding the parameters w that make the data we observed as probable as possible.

If we take the natural logarithm of our Gaussian probability (the log-likelihood), the math simplifies beautifully:

$$\ln p(t|x, w) = -\ln(\sqrt{2\pi}\sigma) - \frac{1}{\sigma^2} \left[\frac{1}{2}(t - y(x, w))^2 \right]$$

Insight: Look closely at the right side of that equation. The first term is a constant. The multiplier $1/\sigma^2$ is a constant. What remains inside the brackets is exactly the **Squared Error**! Because there is a negative sign in front of it, *maximizing* this log-likelihood is mathematically identical to *minimizing* the sum of squared errors.

This is a massive takeaway: **Whenever you minimize Mean Squared Error in your code, you are implicitly assuming your data has Gaussian noise.** If your data has wild outliers (which are highly unlikely under a normal distribution), MSE will punish your model severely.

Basis Functions in Linear Regression

1. Basis Functions: Bending the Line

A standard linear model, like $y(x, w) = w_0 + w_1x$, represents a straight line. However, real-world data is rarely a perfectly straight line.

To capture non-linear, complex relationships while keeping the underlying mathematics simple, we introduce **Basis Functions**, denoted as $\phi_j(x)$. We transform our inputs before multiplying them by our weights.

The model becomes:

$$y(x, w) = w_0\phi_0(x) + w_1\phi_1(x) + \dots + w_M\phi_M(x) = \phi(x)^T w$$

General linear regression model:

0

x 1

$$y(x, \mathbf{w}) = w_1 \phi_1(x) + w_2 \phi_2(x) + \dots + w_p \phi_p(x)$$

where $\phi_j(x)$ are called the basis functions. Can be rewritten as dot product

$$y(x, \mathbf{w}) = \phi(x) \cdot \mathbf{w} = \phi(x)^T \mathbf{w}$$

between weight vector $\mathbf{w} \in \mathbb{R}^p$ and feature vector $\phi(x) = [\phi_1(x), \dots, \phi_p(x)]^T$

This way, we can:

- Keeping model linear in parameters allows simple learning algorithm
- Introducing non-linear transformations of inputs allows to model complex relationships between X and T
- What basis function to choose, again falls under Model Selection

Here are common choices for $\phi_j(x)$:

- **Polynomial:** $\phi_j(x) = x^j$ (e.g., x, x^2, x^3).
- **Radial Basis Functions (RBF/Gaussian):** $\phi_j(x) = \exp\left(-\frac{\|x - \mu_j\|^2}{2s_j^2}\right)$.

□ Linear basis functions

$$\phi_j(x) = x_j$$

□ Polynomial basis functions

$$\begin{aligned} \phi_j(x) &= x^j \\ \phi_j(x) &= x_1^{k_1} x_2^{k_2} \dots x_d^{k_d} \end{aligned}$$

1-dimensional inputs
general

□ Radial (Gaussian) basis functions

$$\phi_j(x) = \exp\left(-\frac{\|x - \mu_j\|^2}{2s_j^2}\right)$$

□ Others, e.g., sigmoidal or hyperbolic

Insight: This is a crucial concept. Why is it still called a "Linear" Regression model if we are fitting curves? Because it remains **linear with respect to the parameters w** . The w values are still just being multiplied and added. This guarantees that our error surface remains a nice, convex bowl with a single global minimum, allowing our algorithms to easily find the best fit.

2. Solving for the Best Fit: The Normal Equations

How do we find the specific weights w that minimize the sum of squared errors $E(w)$?

In calculus, you find the minimum of a function by taking its derivative and setting it to zero. For our multidimensional weights, we take the partial derivative of $E(w)$ with respect to each weight w_i and set the gradient to zero.

Derivative criterion

$$\nabla E(w) = \begin{bmatrix} \frac{\partial E}{\partial w_1} \\ \vdots \\ \frac{\partial E}{\partial w_p} \end{bmatrix} = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}$$

Partial derivatives

$$\frac{\partial}{\partial w_i} E(w) = \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{n=1}^N (t_n - y(x_n, w))^2$$

Can rewrite partial derivative as dot or matrix product

$$\begin{aligned}\frac{\partial}{\partial w_i} E(w) &= - \sum_{n=1}^N (t_n - w \cdot \phi(x_n)) \phi_i(x_n) \\ &= -(t - \Phi w) \cdot \Phi_i \\ &= -\Phi_i^T (t - \Phi w)\end{aligned}$$

residual

residual vector

$$\Phi = \begin{bmatrix} \phi_1(x_1) & \phi_2(x_1) & \cdots & \phi_p(x_1) \\ \phi_1(x_2) & \phi_2(x_2) & \cdots & \phi_p(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(x_N) & \phi_2(x_N) & \cdots & \phi_p(x_N) \end{bmatrix}$$

feature matrix

$$\Phi_i = \begin{bmatrix} \phi_i(x_1) \\ \phi_i(x_2) \\ \vdots \\ \phi_i(x_N) \end{bmatrix}$$

feature column

$$t = \begin{bmatrix} t_1 \\ t_2 \\ \vdots \\ t_N \end{bmatrix}$$

target vector

Gradient criterion becomes

$$\begin{aligned}\nabla E(w) &= -\Phi^T (t - \Phi w) = 0 \\ \Phi^T \Phi w &= \Phi^T t \\ w &= (\Phi^T \Phi)^{-1} \Phi^T t\end{aligned}$$

normal equations

system of linear equations
can be solved by
linear algebra software

Through matrix algebra, this yields the **Normal Equations**:

$$w = (\Phi^T \Phi)^{-1} \Phi^T t$$

- Φ (Phi) is the **Feature Matrix** (or design matrix), where every row is a data point passed through your basis functions.
- t is your vector of target values.

Insight: In theory, the Normal Equation is beautiful—you just crunch the matrices and get the perfect parameters instantly. In practice, as a Machine Learning Engineer, you must be careful. **Matrix inversion**—the $(\Phi^T \Phi)^{-1}$ part—is extremely computationally expensive (roughly $O(p^3)$ time complexity). If you have millions of data points or thousands of features, using the Normal Equations will cause your system to crash or take days to run.

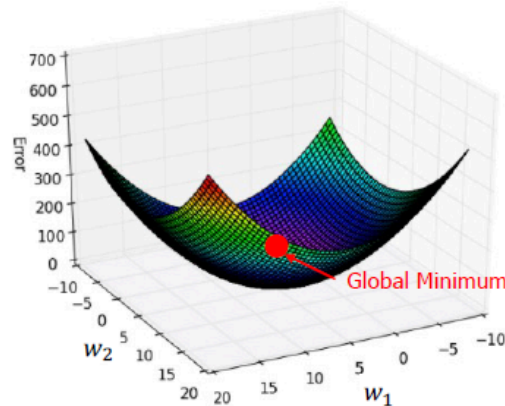
- Fitting criteria: find weights that minimise sum of squared errors
- Is equivalent to maximum likelihood assuming Gaussian noise
- How to identify minimum: derivative criterion

- For linear model with squared error: partial derivative i is dot product of feature column i with residual vector
- Derivative criterion: system of linear equations (normal equations)
- Can be solved by linear algebra algorithm (elimination or iterative)

3. The Workhorse: Gradient Descent

Because the Normal Equations don't scale well to massive datasets and sometimes, other models or loss function may be used, we need a more general, iterative algorithm: **Gradient Descent**.

Gradient descent



How to compute downhill direction?

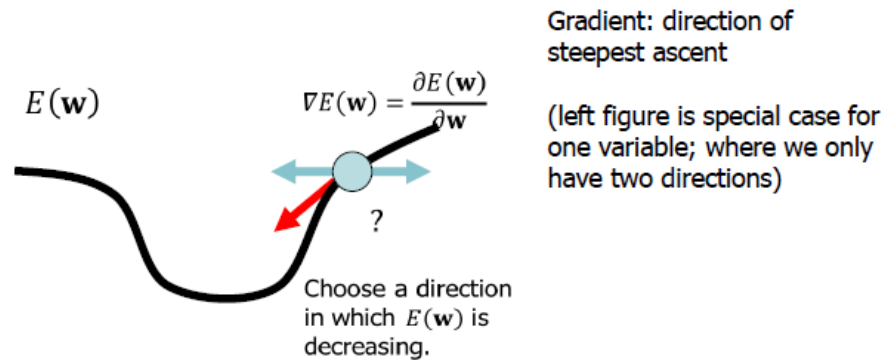
negative gradient of error at current point:

$$-\nabla E(w)$$

Idea: start from a random location on surface and move downhill to find point with lowest error.

Imagine you are standing on a hilly landscape (the error surface) in the fog. To find the lowest point (the global minimum error), you feel the slope of the ground with your foot and take a step downhill.

What does it mean to move downhill on the error function?
That is, how to change the parameter vector $\mathbf{w}$ to improve $E(\mathbf{w})$?



The negative gradient, $-\nabla E(w)$, always points in the direction of steepest descent. The algorithm updates the weights iteratively:

$$\mathbf{w}^{(t)} = \mathbf{w}^{(t-1)} - \eta \nabla E(\mathbf{w}^{(t-1)})$$

Iterative **Algorithm**:

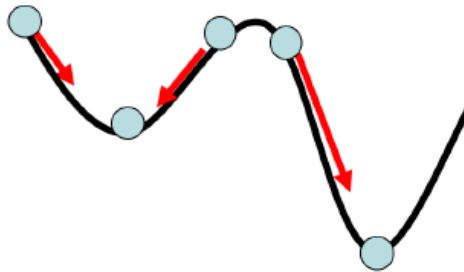
```
choose initial weights  $\mathbf{w}^{(0)}$ 
for  $t = 1, 2, \dots$ 
     $\mathbf{w}^{(t)} = \mathbf{w}^{(t-1)} - \eta \nabla E(\mathbf{w}^{(t-1)})$ 
    break when some condition is met
```

Eta is step-size or
learning rate

- η (eta) is the **learning rate**. It dictates how big of a step you take. If it's too small, training takes forever; if it's too large, you might jump right over the minimum.

How to choose initial weights?

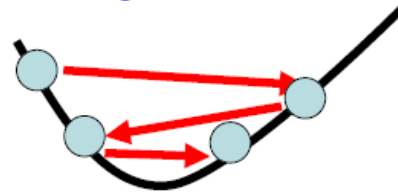
- sensitive decision: local minima
- not an issue for convex errors (like sum of squared errors)



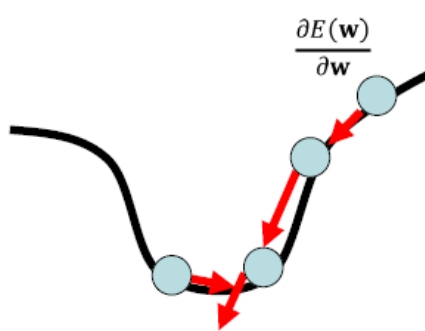
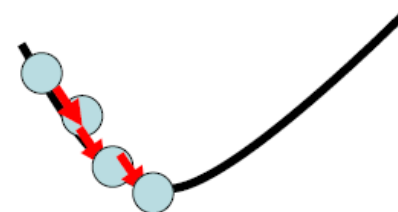
How to choose learning rate?

- static or change with time

Too large?



Too small?



choose initial weights $\mathbf{w}^{(0)}$

for $t = 1, 2, \dots$

$$\mathbf{w}^{(t)} = \mathbf{w}^{(t-1)} - \eta \nabla E(\mathbf{w}^{(t-1)})$$

break when $|E(\mathbf{w}^{(t)}) - E(\mathbf{w}^{(t-1)})| \leq \epsilon$

Could also stop based on

- change in weights
- magnitude of gradient
- ...

4. Scaling Up: Stochastic Gradient Descent (SGD)

Standard Gradient Descent has a flaw: it calculates the gradient using the *entire* training dataset for every single step. This is slow and wasteful for large datasets N . And we also may not have access to all the data at the same time

Stochastic Gradient Descent (SGD) fixes this by calculating the gradient using only a **small, random batch** of data points at each step. Use approximate gradients (based on sample of training data)

Idea

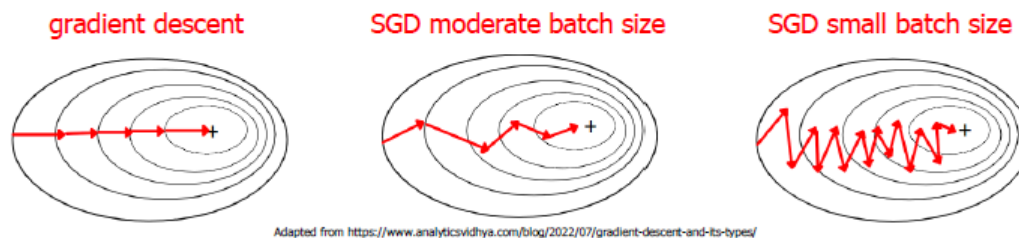
- In each step, use **approximate gradients**
- ...based on small **random batch** of data, say with indices $n_1, \dots, n_b$

$$\begin{aligned}\nabla E(\mathbf{w}) &= -\Phi^T(\mathbf{t} - \Phi\mathbf{w}) \\ &= -\sum_{n=1}^N (t_n - \mathbf{w} \cdot \phi(x_n))\phi(x_n) \\ &\approx -\sum_{i=1}^b (t_{n_i} - \mathbf{w} \cdot \phi(x_{n_i}))\phi(x_{n_i}) \\ &= -\tilde{\Phi}^T(\tilde{\mathbf{t}} - \tilde{\Phi}\mathbf{w}) \\ &= \tilde{\nabla}E(\mathbf{w}) \leftarrow \text{stochastic gradient}\end{aligned}$$

Observations

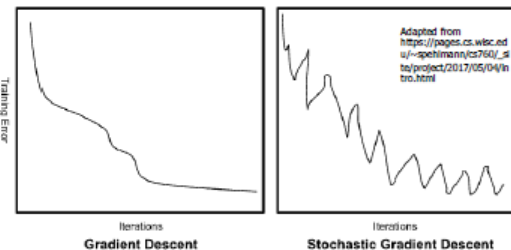
- Can choose **batch size** b arbitrary small and still have $\mathbb{E}[\tilde{\nabla}E(\mathbf{w})] = \nabla E(\mathbf{w})$
 - Can make individual updates very cheap. What is the **downside**?
- **Pros:** Individual updates are computationally very cheap and fast.
 - **Cons:** Because you are only looking at a small slice of data, your path to the minimum becomes "noisy" or zigzagged. A specific step might not even point exactly downhill, but *on average*, it moves toward the minimum.
 - but the expected value of the stochastic error should be the same as the actual error of the dataset - with a distribution

- **Correct mean** of random directions: $\mathbf{E}[\tilde{\nabla}E(\mathbf{w})] = \nabla E(\mathbf{w})$
- ...but the smaller the batch size the more **variance** of directions
- ...consequently might need **more iterations** to converge



Adapted from <https://www.analyticsvidhya.com/blog/2022/07/gradient-descent-and-its-types/>

- In fact, negative stochastic gradient might not even be a **descent direction**
- **Trade-off** when choosing b



Adapted from https://pages.cs.wisc.edu/~seshimann/cs760/_site/project/2017/05/04/intro.html

Handling Overfitting - Regularization

1. The Symptoms of Overfitting - Parametric Regression

Given

- **data** $D = (x_1, t_1), \dots, (x_N, t_N)$
- sampled according to **joint distribution** of RVs X and T
- **input variable(s)** X often assumed with $D_X = \mathbb{R}^d$
- real-valued **target variable** $T \in \mathbb{R}$

Goal / process

- learn **parameters** w of prediction function $y(x, w)$
- ...with low **generalisation error** $\mathbf{E}[(y(X, w) - T)^2]$
- ...via minimisation of **training error** $\sum_{n=1}^N \frac{1}{2n} (t_n - y(x_n, w))^2$

Problem

- We know that training error is not a good estimator for generalisation error

When you use a highly complex model (like a 9th-order polynomial, where $M = 9$), it becomes so flexible that it bends to perfectly pass through every single

training point.

- **The Result:** The training error drops to near zero, but the generalization error (test error) becomes massive.
- **The Symptom:** If you look at the parameters of an overfitted model, the weights are extraordinarily large (e.g., some weights explode to hundreds of thousands). The model is contorting itself violently, using massive positive and negative weights to cancel each other out just to hit specific data points.
- Could weights that do not minimise training error be better?

2. The Cure: Regularization

If the problem is exploding weights, the solution is to punish the model for having them. We do this by changing our objective function. Instead of just minimizing the training error, we minimize the training error *plus* a penalty term for large weights.

The new objective becomes:

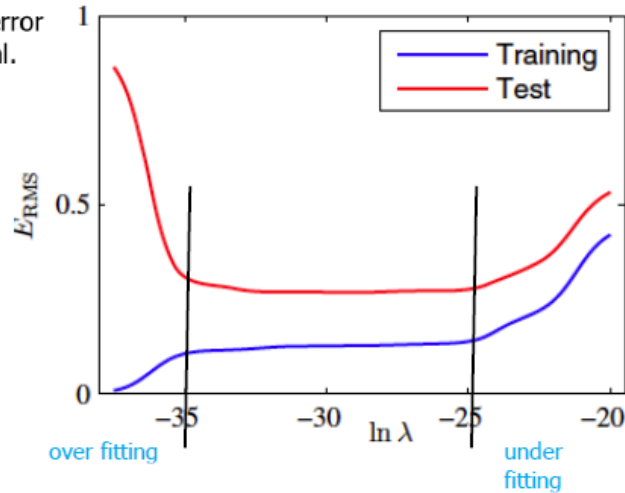
$$E(w) + \lambda\Omega(w)$$

- $E(w)$ is the standard training error.
- $\Omega(w)$ is the penalty function based on the size of the weights.
- λ (lambda) is the **regularization parameter**, a tuning knob that controls how strictly you want to penalize the model.

The λ Trade-off:

Graph of the root-mean-square error vs. $\ln(\lambda)$ for the $M = 9$ polynomial.

- 1 λ increases, model becomes less complex, training error increases
- 2 λ is very small, model too complex, test error high (over fitting)
- 3 λ increases, model becomes less complex, testing error decreases
- 4 λ is too large, model too simple, testing error increases (under fitting)



- **If λ is too large:** The penalty dominates. The model is forced to keep weights near zero, making it too simple. This causes high training and testing error (**Underfitting**).
- **If λ is too small:** The penalty is effectively ignored. The model remains too complex, causing high testing error (**Overfitting**).
- The optimal solution lies in the middle, balancing complexity and fit.

Insight: Finding the right λ is called "Hyperparameter Tuning." In practice, you will never know the best λ in advance. You have to train multiple models with different λ values and use **K-fold Cross-Validation** to see which one performs best on hold-out data.

3. Two Flavors of Penalty: Ridge vs. Lasso

Before applying these penalties, you **must normalize your features** so they are all on the same scale (e.g., don't measure one variable in meters and another in millimeters). If you don't, the penalty will unfairly crush the weights of features that naturally have smaller numerical scales.

There are two primary ways to calculate the penalty $\Omega(w)$:

Ridge Regression

$$\frac{1}{2} \sum_{n=1}^N (t_n - y(x_n, \mathbf{w}))^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

squared Euclidian (L2) norm

$$\|\mathbf{w}\|_2 = \sqrt{\sum_{j=1}^p w_j^2}$$

Lasso Regression

$$\frac{1}{2} \sum_{n=1}^N (t_n - y(x_n, \mathbf{w}))^2 + \frac{\lambda}{2} \|\mathbf{w}\|_1$$

Manhattan (L1) norm

$$\|\mathbf{w}\|_1 = \sum_{j=1}^p |w_j|$$

A. Ridge Regression (L2 Regularization)

- **Math:** Penalizes the sum of the *squared* weights: $\frac{\lambda}{2} \|\mathbf{w}\|_2^2$.
- **Effect:** It shrinks all parameters down to reduce model complexity and prevent overfitting, but it rarely forces any parameter to exactly zero.

B. Lasso Regression (L1 Regularization)

- **Math:** Penalizes the sum of the *absolute values* of the weights: $\frac{\lambda}{2} \|\mathbf{w}\|_1$.
- **Effect:** This is the magical one. Alongside shrinking parameters, Lasso performs **Feature Selection**. If a feature is not truly useful, Lasso will push its weight to *exactly zero*.

Professor's Insight: This is a classic ML interview question. If you have a dataset with 10,000 features (like genetic data or text data) and you know most of them are noise, use Lasso. It will zero out the garbage, leaving you with a sparse, fast, and highly interpretable model. If all your features are relatively useful and you just want to prevent overfitting, use Ridge.

4. Optimization Quirks - using SGD

How does this change our Gradient Descent and Normal Equations?

For ridge regularised error is still a **quadratic function** of weights

$$\frac{1}{2} \sum_{n=1}^N (t_n - y(x_n, \mathbf{w}))^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

Finding gradient and setting to 0 again identifies system of **linear equations** with solution:

$$\mathbf{w} = (\lambda \mathbf{I} + \Phi^T \Phi)^{-1} \Phi^T \mathbf{t}$$

Hence, can be solved directly or iteratively with (S)GD

Sidenote: Can derive ridge regression as **Bayesian method** by defining normal distribution prior distribution on parameters

- **Ridge Optimization:** The squared penalty keeps the math smooth and differentiable. We can adapt the Normal Equations directly to find the exact solution: $\mathbf{w} = (\lambda \mathbf{I} + \Phi^T \Phi)^{-1} \Phi^T \mathbf{t}$. It can also be easily solved with Gradient Descent.

Problem: Lasso regularised error is **not differentiable**

$$\frac{1}{2} \sum_{n=1}^N (t_n - y(x_n, \mathbf{w}))^2 + \frac{\lambda}{2} \|\mathbf{w}\|_1$$

However, is still **convex**.

Popular options:

- ☐ Use gradient descent anyway with "sub-gradient"
- ☐ Coordinate-wise descent

- **Lasso Optimization:** Because the absolute value function has a sharp "V" shape at zero, it is not strictly differentiable at zero. You cannot use standard calculus. Instead, we have to use specialized iterative algorithms like **Coordinate Descent** or sub-gradient methods.

What are key differences between Ridge and Lasso regression

- Ridge: The main advantage is parameter shrinkage and reduce the model complexity
- Lasso: Along with shrinking parameters, it selects input variables as well. Some features can be excluded from the model.

When do we have to use Ridge or Lasso?

- Ridge: Mainly used to prevent overfitting, but not useful in case of high # of input variables.
- Lasso: It provides sparse solutions, but useful in case of high # of input variables as those ones with zero parameters can simply be ignored.

L1 : feature selection?

Shoe size	Weight	Height
2	1.8	4
3	3.1	6
...	...	...
4	3.9	8

h: height, the target variable

s: shoe size

w: weight

$$h = \alpha_1 s + \alpha_2 w$$



$$\alpha_1 + \alpha_2 = 2$$

$$\alpha_1=1, \alpha_2 = 1 \text{ or } \alpha_1 = 2, \alpha_2 = 0 \text{ or } \alpha_1 = 0, \alpha_2 = 2$$

Shoe size	Weight	Height
2	1.8	4
3	3.1	6
...	...	...
4	3.9	8

h: height, the target variable

s: shoe size

w: weight

$$h = \alpha_1 s + \alpha_2 w$$

$$E = \sum_{i=1}^n (h' - \alpha_1 s - \alpha_2 w)^2 + \lambda (\alpha_1^2 + \alpha_2^2) \xrightarrow{\quad\quad\quad} \alpha_1 = 1, \alpha_2 = 1$$

$$E = \sum_{i=1}^n (h' - \alpha_1 s - \alpha_2 w)^2 + \lambda (|\alpha_1| + |\alpha_2|)$$



$$\alpha_1 = 2, \alpha_2 = 0 \text{ or } \alpha_1 = 0, \alpha_2 = 2$$